Homework #4

Due: April 3, 2026 at 11:59pm

Representation Learning, Transformers, and Non-parametric methods

Introduction

This homework contains three problems:

1. You will do some transformer computations by hand, and implement self-attention from scratch and produce attention score heatmaps.
2. You will do some derivations related to autoencoders and implement them.
3. You will do some math to prove bounds on decision tree ensembles, in order to

We note that this homework does not have any problem where you directly run through the computations you would do to derive a decision tree. Doing so can be good review, however, so for an example with those computations, please see the last exercise in Section 6 notes.

Resources and Submission Instructions

Please type your solutions after the corresponding problems using this LaTeX template, and start each main problem on a new page.

Please submit the writeup PDF to the Gradescope assignment 'HW4'. Remember to assign pages for each question. **You must include any plots in your writeup PDF.** . Please submit your LaTeXfile and code files to the Gradescope assignment 'HW4 - Supplemental.' The supplemental files will only be checked in special cases, e.g. honor code issues, etc. Your files should be named in the same way as we provide them in the repository, e.g. `hw4.pdf`, etc.

Problem 1 (Understanding the Transformer, 40 pts)

Transformers are the dominant architecture behind large language models and many modern sequence processing systems. At the heart of every transformer is **self-attention**, a mechanism that allows each position in a sequence to "look at" every other position and decide how much information to gather from each.

In this problem you will (a) work through the math of scaled dot-product attention, (b) understand key design choices, and (c) implement the attention layer.

Background. Given an input matrix $\mathbf{X} \in \mathbb{R}^{T \times d}$ (one row per token), self-attention computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V},$$

where $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ with learnable weight matrices $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$. The softmax is applied row-wise: for a row vector $\mathbf{z} \in \mathbb{R}^T$, $\text{softmax}(\mathbf{z})_i = e^{z_i} / \sum_{j=1}^{T} e^{z_j}$.

1. **[10 pts] Attention by hand.**

   Consider a sequence of $T = 2$ tokens with embedding dimension $d = 2$. The input matrix, query weight matrix, key weight matrix, and value weight matrix are:

   $$\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{W}_Q = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{W}_K = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{W}_V = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}.$$

   So $d_k = d_v = 2$.

   (a) Compute $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$, and the raw score matrix $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$.

   (b) Compute the scaled score matrix $\mathbf{S}/\sqrt{d_k}$. Then apply softmax row-wise to obtain the attention weight matrix $\mathbf{A} \in \mathbb{R}^{2 \times 2}$.

   You may leave your answer in terms of $e^{(\cdot)}$ and sums, or compute decimal approximations.

2. **[5 pts] Why scale by $\sqrt{d_k}$?**

   Suppose the entries of $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{d_k}$ are independent random variables with mean 0 and variance 1.

   (a) Compute the mean and variance of the dot product $\mathbf{q}^\top\mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$.

   (b) As d_k grows, what happens to the magnitude of $\mathbf{q}^\top\mathbf{k}$? Explain in one or two sentences what effect this has on the softmax distribution (i.e. does it become more uniform or more peaked?).

   (c) Show that after scaling by $1/\sqrt{d_k}$ the dot product has variance 1 regardless of d_k.

3. **[5 pts] Permutation equivariance.**

   Let $\mathbf{P}_\sigma \in \mathbb{R}^{T \times T}$ be the permutation matrix corresponding to a permutation σ of $\{1, \ldots, T\}$ (i.e. $(\mathbf{P}_\sigma)_{ij} = \mathbf{1}[j = \sigma(i)]$).

   (a) Show that self-attention (without positional encoding) is **permutation equivariant**: if we permute the input rows by replacing $\mathbf{X}$ with $\mathbf{P}_\sigma\mathbf{X}$, the output is permuted in the same way. That is, show

   $$\text{Attention}(\mathbf{P}_\sigma\mathbf{X}) = \mathbf{P}_\sigma \, \text{Attention}(\mathbf{X}).$$

   Hint: What happens to $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$, and the softmax when you left-multiply $\mathbf{X}$ by $\mathbf{P}_\sigma$?

(b) Explain in two or three sentences why permutation equivariance is a problem for tasks like language modeling, where word order matters.

(c) The standard fix is to add a **positional encoding PE** $\in \mathbb{R}^{T\times d}$ to the input: $\mathbf{X}' = \mathbf{X} + \mathbf{PE}$. Why does this break permutation equivariance?

4. **[10 pts] Implementing self-attention from scratch (code).**

   In the accompanying notebook, implement single-head scaled dot-product self-attention using **only NumPy** (do not use pre-existing implementations such as PyTorch and sklearn).

   Your function should take:

   - $\mathbf{X} \in \mathbb{R}^{T\times d}$: input embeddings
   - $\mathbf{W}_Q \in \mathbb{R}^{d\times d_k}$, $\mathbf{W}_K \in \mathbb{R}^{d\times d_k}$, $\mathbf{W}_V \in \mathbb{R}^{d\times d_v}$: weight matrices

   and return:

   - output $\in \mathbb{R}^{T\times d_v}$: the attention output
   - weights $\in \mathbb{R}^{T\times T}$: the attention weight matrix $\mathbf{A}$

   Implementation tips:

   - For numerical stability, before applying softmax to a row $\mathbf{z}$, subtract $\max(\mathbf{z})$: compute $\mathrm{softmax}(\mathbf{z} - \max(\mathbf{z}))$. (As an aside, note that this does not change the softmax function.)

5. **[10 pts] Multi-head attention**

   In the notebook, extend your implementation to **multi-head attention** and use it inside a small transformer classifier trained on a synthetic sequence dataset.

   Given h heads, each head uses its own $\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)} \in \mathbb{R}^{d\times d_k}$ and $\mathbf{W}_V^{(i)} \in \mathbb{R}^{d\times d_v}$, where $d_k = d_v = d/h$. The outputs of all heads are concatenated and projected:

$$\mathrm{MultiHead}(\mathbf{X}) = [\mathrm{head}_1\,;\,\cdots\,;\,\mathrm{head}_h]\,\mathbf{W}_O,$$

   where $\mathbf{W}_O \in \mathbb{R}^{d\times d}$.

   After training, attach one learned attention matrix as a heatmap and briefly describe what it suggests the model is attending to.

Solution:

1(a)(i). Row wise multiplication gives

$$Q = XW_Q = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad K = XW_K = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad V = XW_V = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$$

The raw scores are $S_{ij} = q_i^\top k_j$ (dot product of row i of Q with row j of K):

$$S = QK^\top = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$$

1(a)(ii). With $d_k = 2$, the scaled matrix is $S/\sqrt{d_k}$:

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1/\sqrt{2} & \sqrt{2} \\ 0 & 1/\sqrt{2} \end{pmatrix}$$

For row i, subtract the row max, exponentiate, and normalize. For row 1 the max is $\sqrt{2}$ at column 2, so the shifted row is $[-1/\sqrt{2}, 0]$ and

$$A_{1j} = \frac{e^{z_j - \max}}{\sum_k e^{z_k - \max}}, \quad A_{11} = \frac{e^{-1/\sqrt{2}}}{1 + e^{-1/\sqrt{2}}}, \quad A_{12} = \frac{1}{1 + e^{-1/\sqrt{2}}}$$

Row 2 has the same shifted values $[-1/\sqrt{2}, 0]$, hence the same attention weights as row 1. Numerically $A_{11} \approx 0.33$, $A_{12} \approx 0.67$. The attention output is AV (each row of A times V).

1(b)(i). By linearity, $\mathbb{E}[q^\top k] = \sum_i \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$. Because $q_i k_i$ are independent with mean 0,

$$\mathrm{Var}(q^\top k) = \sum_{i=1}^{d_k} \mathrm{Var}(q_i k_i) = \sum_{i=1}^{d_k} \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = d_k$$

1(b)(ii). The typical magnitude of $q^\top k$ grows like $\sqrt{d_k}$ (standard deviation $\sqrt{d_k}$). Large dot products drive softmax inputs into large positive and negative values, so the distribution becomes more peaked (nearly one-hot).

1(b)(iii). Scaling gives $(q/\sqrt{d_k})^\top k = (q^\top k)/\sqrt{d_k}$, which has variance $\mathrm{Var}(q^\top k)/d_k = 1$.

1(c)(i). Let $\widetilde{X} = P_\sigma X$. Then $\widetilde{Q} = P_\sigma Q$, $\widetilde{K} = P_\sigma K$, $\widetilde{V} = P_\sigma V$. The score matrix becomes $\widetilde{S} = \widetilde{Q}\widetilde{K}^\top = P_\sigma S P_\sigma^\top$, so row i of $\widetilde{S}$ is row $\sigma(i)$ of S with columns permuted the same way. Softmax is applied row-wise, so $\widetilde{A} = P_\sigma A P_\sigma^\top$. Then $\widetilde{A}\widetilde{V} = P_\sigma AV$. Thus $\mathrm{Attention}(P_\sigma X) = P_\sigma \,\mathrm{Attention}(X)$.

1(c)(ii). Permutation equivariance means the layer cannot tell which position is first or second, shuffling words shuffles outputs the same way. Language meaning depends on order, so the model must distinguish positions.

1(c)(iii). The positional matrix PE assigns different vectors to different positions. Permuting rows of X without permuting the same rows of PE changes which position gets which encoding, so the input to attention is no longer a pure row-permutation of the original; equivariance fails.

1(d). Implemented in `hw4_release.ipynb`, with row wise max subtracted softmax.

1(e). Training curves and a learned attention map are shown below. The model reaches high test accuracy on the synthetic task. The heatmap for head 0 suggests the `[CLS]` query places substantial mass on the position containing `VAL0`/`VAL1`, which carries the label, rather than on distractor noise tokens.

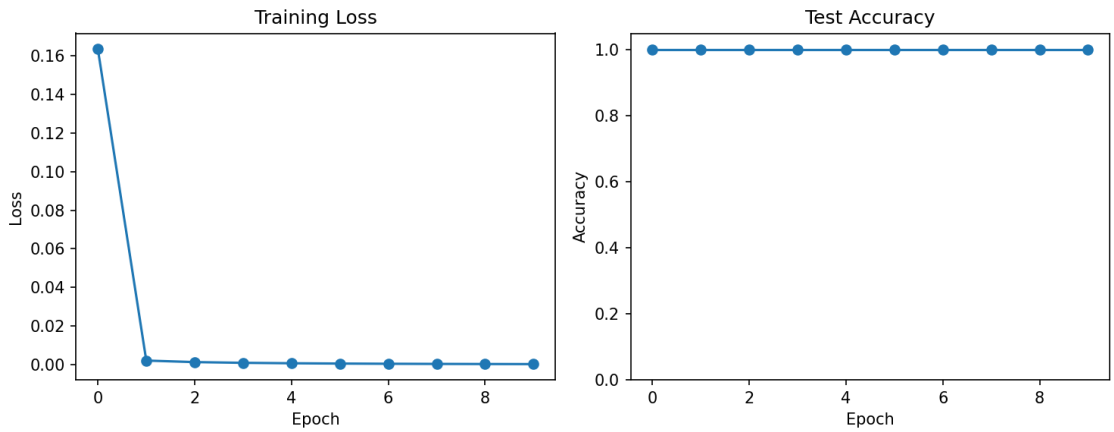

Figure 1: Synthetic transformer: training loss and test accuracy.

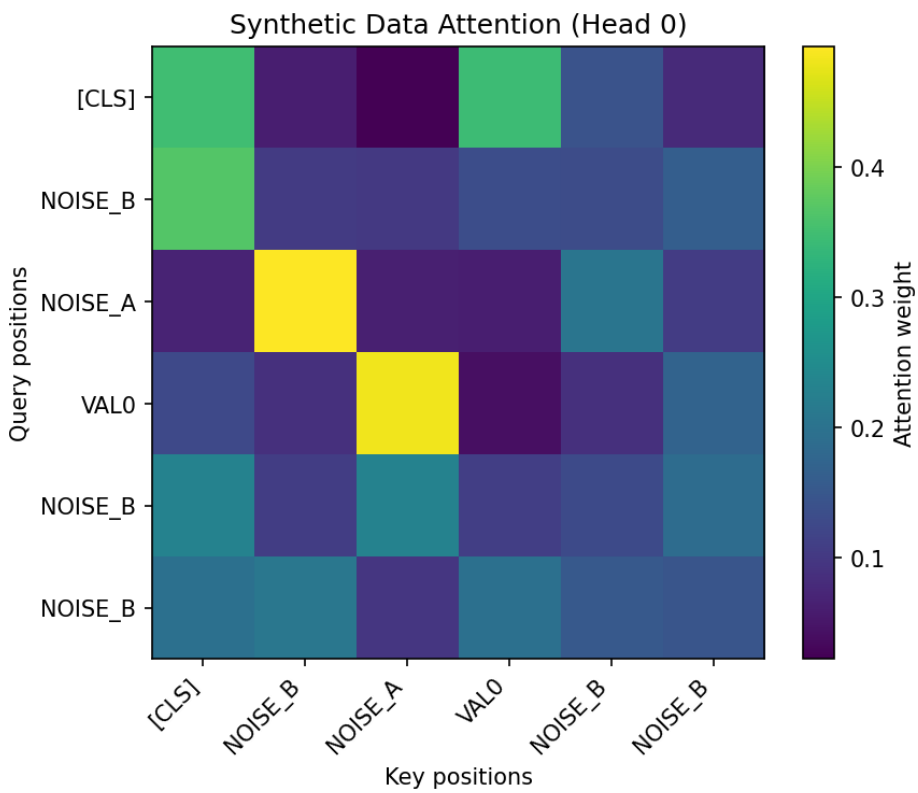

Figure 2: Attention weights (head 0) on one test sequence.

Problem 2 (Autoencoders, 76 pts)

Suppose we want to compress our data into a lower-dimensional representation. What is a principled way to do this with machine learning, such that we can be confident the compression retains as much useful information as possible?

1. The Autoencoder.

   We can turn this into a game between two networks. An *encoder* $f_\phi : \mathbb{R}^n \to \mathbb{R}^d$ takes an input $\mathbf{x}$ and compresses it down to d dimensions: $\mathbf{z} = f_\phi(\mathbf{x})$. A *decoder* $g_\theta : \mathbb{R}^d \to \mathbb{R}^n$ must then reconstruct the original input from just this compressed representation: $\hat{\mathbf{x}} = g_\theta(\mathbf{z})$. We measure how well the decoder succeeds by comparing $\hat{\mathbf{x}}$ to $\mathbf{x}$, and backpropagate through both networks:

   $$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}_i - g_\theta(f_\phi(\mathbf{x}_i))\|^2.$$

   This is the *reconstruction loss*, and it is the core of how autoencoders are trained. The logic is simple: if the decoder can faithfully recover $\mathbf{x}$ from $\mathbf{z}$, then $\mathbf{z}$ must have captured the essential structure of $\mathbf{x}$.

   Note that what makes a model an "autoencoder" is this encoder–decoder coupling trained with a reconstruction loss—not any particular architecture. The encoder and decoder can be MLPs, CNNs, Transformers, or anything else. It is the training paradigm that defines the autoencoder.

   (a) **Implementation**[12 pts]. In the notebook, implement a convolutional autoencoder on CelebA. Train it, and provide your training and test loss curves. Display the first 5 test images alongside their reconstructions.

   (b) **Sampling**[4 pts]. In the notebook, sample random vectors $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ from the autoencoder's latent space and decode them. Do the outputs look like faces? In 1–2 sentences, explain why the autoencoder has no reason to make its latent space produce coherent outputs when sampled this way.

2. The autoencoder gives us a compressed representation, but its latent space has no structure that would let us *generate* new data. What if, instead of learning point embeddings, we tried to learn the probability distribution over images, and then sample from it?

   The natural objective is maximum likelihood: given a generative model with latent variables $\mathbf{z}$ and a prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, the log-likelihood of a datapoint $\mathbf{x}$ is

   $$\log p(\mathbf{x}) = \log \int p(\mathbf{x}, \mathbf{z})\, d\mathbf{z} = \log \int p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}.$$

   To train this model via maximum likelihood, we would need the true posterior $p(\mathbf{z} \mid \mathbf{x})$. By Bayes' theorem,

   $$p(\mathbf{z} \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})}{p(\mathbf{x})}.$$

   However, computing $p(\mathbf{x})$ (the evidence) requires evaluating the integral above over all possible values of $\mathbf{z}$, which is intractable when $p(\mathbf{x} \mid \mathbf{z})$ is parameterized by a neural network in high dimensions.

(a) **Intractability**[4 pts]. In 1–2 sentences, explain concretely why the integral $\int p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$ is intractable. Why can't we just estimate it by drawing samples $\mathbf{z}^{(k)} \sim p(\mathbf{z})$ and averaging $p(\mathbf{x} \mid \mathbf{z}^{(k)})$?

Since we cannot compute $p(\mathbf{z} \mid \mathbf{x})$ exactly, we introduce a neural network $q_\phi(\mathbf{z} \mid \mathbf{x})$ that serves as an *approximate posterior*—our best learned guess at the true posterior. To measure how well $q_\phi(\mathbf{z} \mid \mathbf{x})$ approximates $p(\mathbf{z} \mid \mathbf{x})$, we use the Kullback–Leibler (KL) divergence and once again need a clever technique:

$$D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z} \mid \mathbf{x})\big) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\Big[\log q_\phi(\mathbf{z} \mid \mathbf{x}) - \log p(\mathbf{z} \mid \mathbf{x})\Big].$$

(b) **Deriving the ELBO.** Starting from the KL divergence above:

i. **Expanding** [8 pts]. Expand $\log p(\mathbf{z} \mid \mathbf{x})$ using Bayes' rule, and rearrange to show:

$$\log p(\mathbf{x}) = D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z} \mid \mathbf{x})\big) + \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\Big[\log p(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z} \mid \mathbf{x})\Big].$$

ii. **Interpretation** [4 pts]. We define the second term to be the **Evidence Lower Bound (ELBO)**:

$$\mathrm{ELBO} = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\Big[\log p(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z} \mid \mathbf{x})\Big].$$

Since $D_{\mathrm{KL}} \geq 0$, conclude that $\mathrm{ELBO} \leq \log p(\mathbf{x})$. In one sentence, explain why this means maximizing the ELBO is a sensible surrogate for maximizing the intractable log-likelihood.

iii. **Rewriting** [8 pts]. Now show that the ELBO can be rewritten as:

$$\mathrm{ELBO} = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p(\mathbf{x} \mid \mathbf{z})\big] - D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big).$$

(Hint: split $\log p(\mathbf{x}, \mathbf{z}) = \log p(\mathbf{x} \mid \mathbf{z}) + \log p(\mathbf{z})$.)

(c) **Interpreting the VAE loss.** The loss we minimize is the negative ELBO:

$$\mathcal{L}_{\mathrm{VAE}} = -\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p(\mathbf{x} \mid \mathbf{z})\big] + D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big).$$

i. **Interpretation**[4 pts]. The first term is the (negative) expected log-likelihood of $\mathbf{x}$ given samples from the encoder. In 1–2 sentences, explain how this connects back to the reconstruction loss from Section 1. What does minimizing it encourage?

ii. **Interpretation**[4 pts]. The second term penalizes the encoder's output distribution $q_\phi(\mathbf{z} \mid \mathbf{x})$ for deviating from the prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$. Explain how this term addresses the problem you identified in question 1(b): why does it give the model an incentive to organize the latent space for generation?

iii. **Tensity**[4 pts]. These two terms pull in opposite directions. In 1–2 sentences, describe the qualitative effect of this tension on the VAE's reconstructions and generated samples, compared to the standard autoencoder.

3. VAE Implementation

In a VAE, the encoder outputs the parameters of the approximate posterior $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \operatorname{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x})))$. That is, a shared convolutional backbone produces a feature vector, which is then projected through two separate linear heads: one outputting $\boldsymbol{\mu} \in \mathbb{R}^d$ and one outputting $\log \boldsymbol{\sigma}^2 \in \mathbb{R}^d$. During training, we sample $\mathbf{z}$ from this distribution; during generation, we sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and decode.

(a) **Reparameterization**[8 pts]. We need to backpropagate through the sampling step $\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{x})$, but sampling is not differentiable. In 1–2 sentences, explain why we cannot directly backpropagate gradients through a sampling operation with respect to the distribution's parameters. Show that if $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then $\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}$ has distribution $q_\phi(\mathbf{z} \mid \mathbf{x})$, and explain briefly why this makes the loss differentiable w.r.t. ϕ.

(b) **Closed-form KL**[6 pts]. Derive the closed-form expression for the KL term when $q = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$ and $p = \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$D_{\mathrm{KL}}(q \| p) = \frac{1}{2} \sum_{j=1}^{d} \Big(\mu_j^2 + \sigma_j^2 - \ln \sigma_j^2 - 1 \Big).$$

(Hint: work coordinate-by-coordinate. Use $\mathbb{E}[z_j] = \mu_j$ and $\mathbb{E}[z_j^2] = \mu_j^2 + \sigma_j^2$.)

(c) **Implementation**[12 pts]. In the notebook, implement a VAE using the architecture described above. Train it, and provide your training and test loss curves (decomposed into reconstruction and KL terms). Display the first 5 test images alongside their reconstructions. Then, sample from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ and decode—do the samples look like faces? Compare to your results from 1(b).

Solution:

1(a). Implemented in the notebook. convolutional encoder-decoder on CelebA with train/test MSE curves below, five test originals vs. reconstructions.

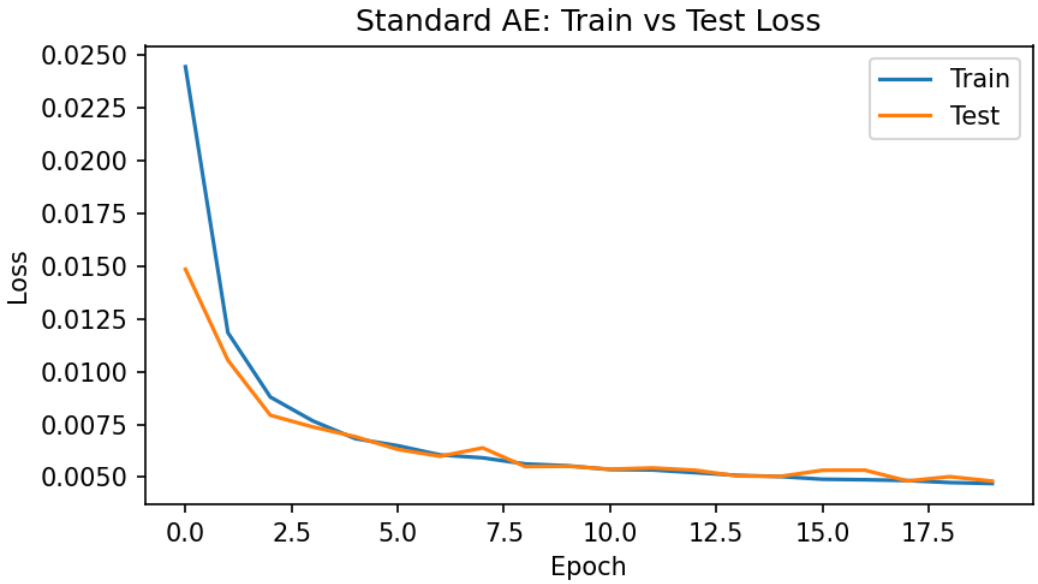

Figure 3: Standard autoencoder: train vs. test reconstruction MSE.

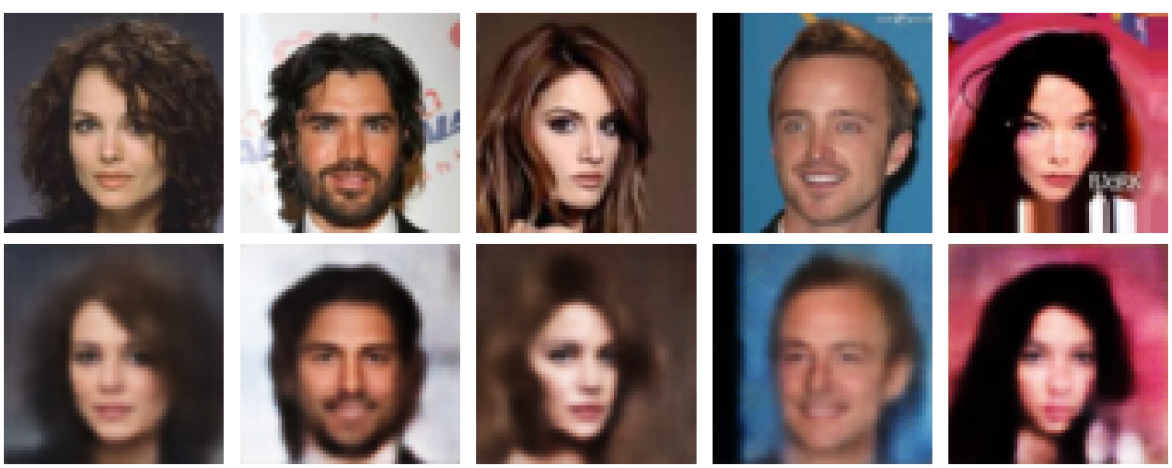

Figure 4: Five CelebA test images (top) and reconstructions (bottom).

1(b). Random $z \sim \mathcal{N}(0, I)$ decoded from the AE usually do not look like realistic faces (see figure). The AE is only trained to map each training image to some code and back, nothing forces those codes to cover $\mathbb{R}^d$ or to match the Gaussian prior, so most Gaussian draws lie in regions the decoder never learned to use.

Figure 5: AE: samples from $\mathcal{N}(0, I)$ passed through the decoder.

2(a). The integral $\int p(x \mid z)\, p(z)\, dz$ is a high-dimensional integral of a highly peaked neural likelihood against the prior; we cannot enumerate a grid. Monte Carlo with $z^{(k)} \sim p(z)$ estimates the average of $p(x \mid z)$, not the integral: the quantity $\mathbb{E}_{p(z)}[p(x \mid z)]$ differs from $p(x)$ by Jensen's gap for log, and the variance of importance style estimators explodes when the prior samples rarely hit regions where $p(x \mid z)$ is large.

2(b)(i). Using Bayes rule, $\log p(z \mid x) = \log p(x \mid z) + \log p(z) - \log p(x)$. Substitute into the KL definition and expand the expectation:

$$D_{\mathrm{KL}}\big(q_\phi \,\|\, p(\cdot \mid x)\big) = \mathbb{E}_q\big[\log q_\phi(z \mid x) - \log p(x \mid z) - \log p(z)\big] + \log p(x)$$

Rearranging yields

$$\log p(x) = D_{\mathrm{KL}}\big(q_\phi \,\|\, p(\cdot \mid x)\big) + \mathbb{E}_q\big[\log p(x, z) - \log q_\phi(z \mid x)\big]$$

2(b)(ii). Since $D_{\mathrm{KL}} \geq 0$, the second term (ELBO) satisfies ELBO $\leq \log p(x)$. Maximizing a lower bound on $\log p(x)$ pushes up the true likelihood whenever the bound is reasonably tight.

2(b)(iii). Split $\log p(x, z) = \log p(x \mid z) + \log p(z)$:

$$\mathrm{ELBO} = \mathbb{E}_q[\log p(x \mid z)] + \mathbb{E}_q[\log p(z) - \log q_\phi(z \mid x)] = \mathbb{E}_q[\log p(x \mid z)] - D_{\mathrm{KL}}\big(q_\phi \,\|\, p(z)\big)$$

2(c)(i). With Gaussian decoder, $\log p(x \mid z)$ is (up to constants) negative squared error, minimizing $-\mathbb{E}_q[\log p(x \mid z)]$ is the same spirit as minimizing reconstruction MSE, encouraging accurate reconstructions from latent draws.

2(c)(ii). Matching q_ϕ to $\mathcal{N}(0, I)$ spreads mass so that sampling $z \sim \mathcal{N}(0, I)$ at test time lands in regions the decoder was trained to invert, addressing the unstructured latent of 1(b).

2(c)(iii). Strong reconstruction pushes informative but possibly irregular codes; the KL pulls codes toward the prior and can blur reconstructions while making samples more coherent than the plain AE.

3(a). A discrete draw $z \sim q_\phi$ does not depend smoothly on (μ, σ) in a way PyTorch can differentiate through. For $\epsilon \sim \mathcal{N}(0, I)$, the vector $z = \mu + \sigma \odot \epsilon$ is Gaussian with mean μ and diagonal variance σ^2; gradients flow through μ and σ while ϵ is a fixed noise draw per forward pass.

3(b). Coordinatewise, $D_{\mathrm{KL}}(q_j \,\|\, p_j) = \frac{1}{2}\big(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\big)$, summing gives the stated formula.

3(c). VAE training curves (reconstruction, KL, total) and visual results are below. Random prior samples are more face like than the AE's because the KL term aligns the aggregate posterior with $\mathcal{N}(0, I)$.

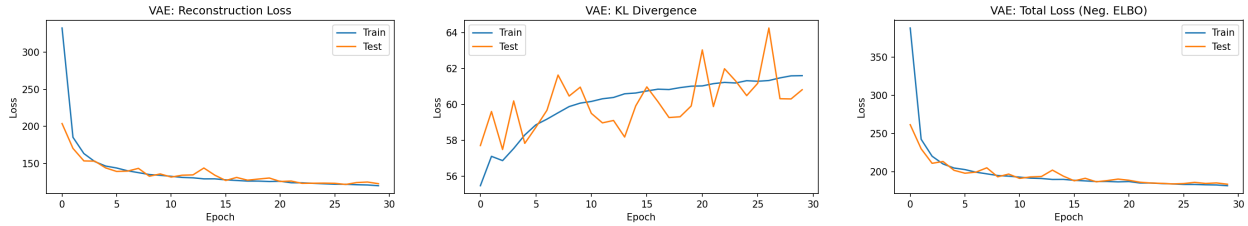

Figure 6: VAE: reconstruction term, KL term, and total negative ELBO (train vs. test).

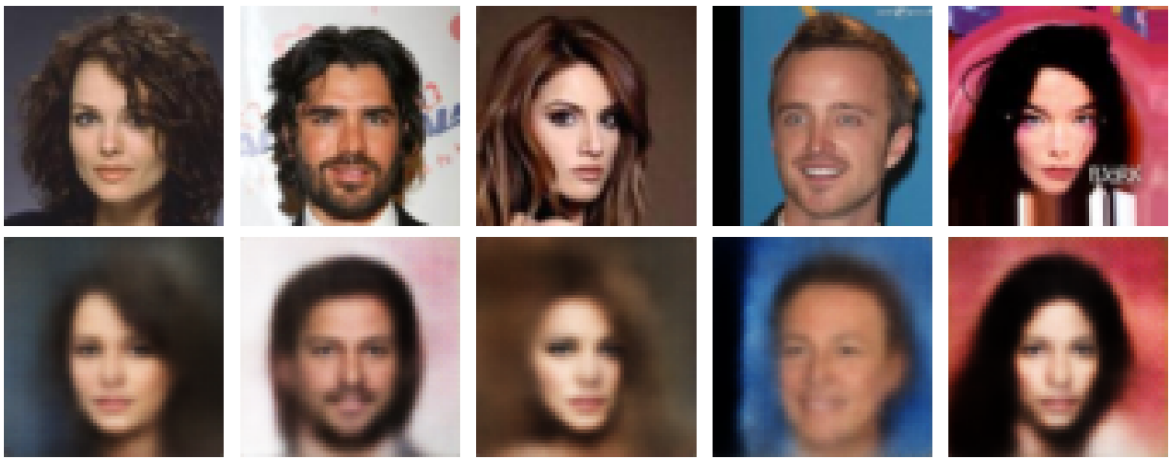

Figure 7: VAE: five test originals (top) and reconstructions (bottom).

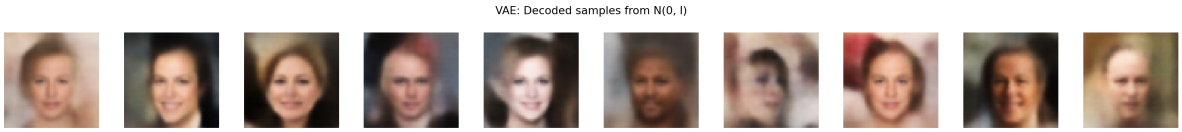

Figure 8: VAE: decoding random $z \sim \mathcal{N}(0, I)$.

Part 4 (latent arithmetic). The notebook computes attribute direction vectors in latent space and applies `apply_attribute` / `interpolate_attribute`. Below are VAE and AE grids (negative / original / positive along each direction), then VAE interpolations along each direction.

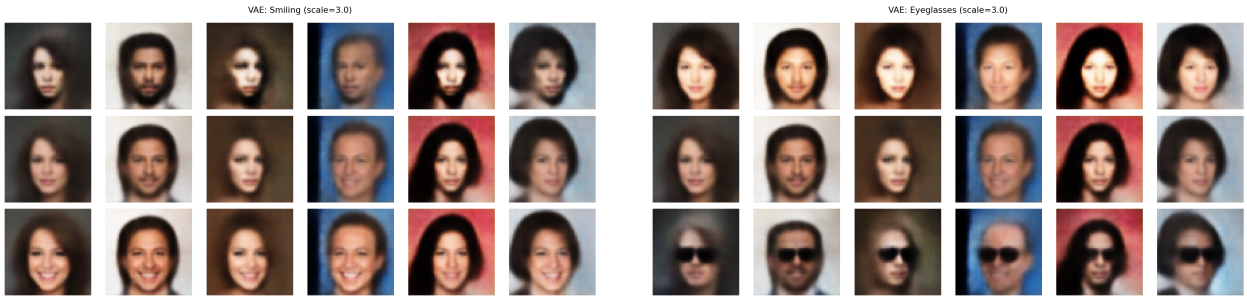

Figure 9: VAE: latent attribute shifts for `Smiling` and `Eyeglasses` (rows: −direction, original, +direction).

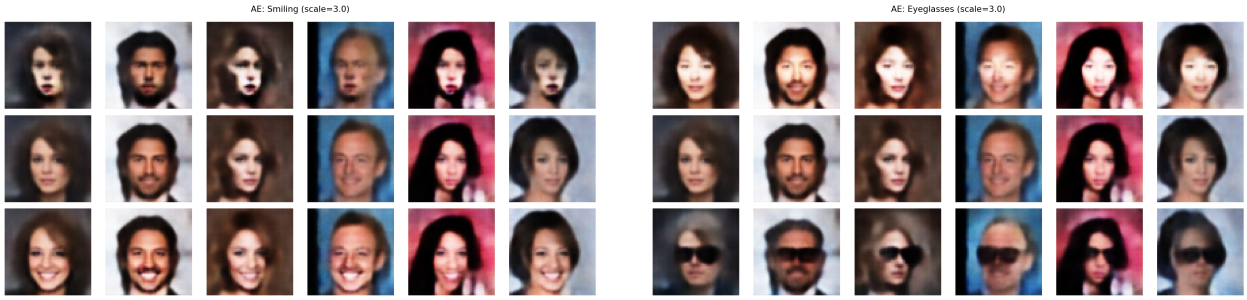

Figure 10: Standard AE: same latent attribute manipulations.

Figure 11: VAE: interpolation along the learned `Smiling` and `Eyeglasses` directions.

Problem 3 (Decision Trees, Random Forests and Mixture of Experts)

1. Suppose we have B independently trained binary classifiers (e.g., decision trees), each with the same accuracy $p > \frac{1}{2}$ on a given test point. The ensemble predicts by **majority vote**.

   Let X be the number of trees that predict correctly. Then $X \sim \text{Binomial}(B, p)$, and the ensemble is correct whenever $X > B/2$.

   Background: Hoeffding's Inequality. Let $Z_1, \ldots, Z_B$ be independent random variables with $Z_i \in [a_i, b_i]$. Hoeffding's inequality states that for the sample mean $\bar{Z} = \frac{1}{B}\sum_{i=1}^{B} Z_i$:

   $$P\big(\bar{Z} - \mathbb{E}[\bar{Z}] \leq -t\big) \;\leq\; \exp\left(\frac{-2B^2t^2}{\sum_{i=1}^{B}(b_i - a_i)^2}\right).$$

   Intuitively, Hoeffding's inequality is a *concentration inequality*: it says that the average of many bounded independent random variables is unlikely to be far from its expectation, and this probability shrinks *exponentially* in the number of samples B. In our setting, each $Z_i \in \{0, 1\}$ indicates whether tree i is correct, so $\bar{Z} = X/B$ and $\mathbb{E}[\bar{Z}] = p$.

   (a) [**5 points**] Apply Hoeffding's inequality to show that the ensemble error rate satisfies

   $$P(X \leq \tfrac{B}{2}) \leq \exp\Big(-2B\big(p - \tfrac{1}{2}\big)^2\Big).$$

   Hint: The ensemble errs when $\bar{Z} \leq \frac{1}{2}$.

   (b) [**5 points**] Compute the bound for $p = 0.6$ and $B \in \{10, 100, 1000\}$. At what B does the bound first drop below 10^{-6}? Is that good or necessary?

   (c) [**5 points**] The proof above assumes the trees are *independent*. In practice, bagged trees are trained on overlapping bootstrap samples and are therefore correlated.

      i. Explain intuitively why correlation weakens the convergence guarantee.

      ii. Random forests add **feature subsampling** (each split considers only m of d features at random). Explain how this reduces correlation and partially restores convergence.

2. Part 1(c) showed that correlation between trees matters. We now make this precise. Suppose we have B trees $T_1, \ldots, T_B$, each with the same prediction variance $\sigma^2 = \text{Var}(T_b(\mathbf{x}))$, and each pair has the same correlation $\rho = \text{Corr}(T_i(\mathbf{x}), T_j(\mathbf{x}))$ for $i \neq j$.

   (a) [**8 points**] Prove that the variance of the ensemble prediction is:

   $$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

   Hint: Expand the variance of the sum, separate the diagonal terms ($i = j$) from the off-diagonal terms ($i \neq j$), and use $\text{Cov}(T_i, T_j) = \rho\,\sigma^2$.

   (b) [**4 points**] Analyze the formula:

      i. What does the ensemble variance converge to as $B \to \infty$? Can we make the variance arbitrarily small by adding more trees?

      ii. What happens when $\rho = 0$? When $\rho = 1$? Relate each case to a practical scenario.

(c) **[4 points]** Random forests use $m = \lfloor \sqrt{d} \rfloor$ features per split (for classification). Using the formula from (d), explain why $m = 1$ (one random feature per split) is generally a bad idea, even though it minimizes ρ. What tradeoff does m control?

3. **Mixture of Experts (MoE).** A random forest is a *dense ensemble*: every tree processes every input. A Mixture of Experts is a *sparse ensemble*: a learned gating network routes each input to only a few specialized sub-networks ("experts"), and the output is a weighted combination:

$$y = \sum_{k=1}^{K} g_k(\mathbf{x})\, E_k(\mathbf{x}), \quad \text{with most } g_k = 0 \text{ (sparse routing).}$$

This is the architecture behind many modern LLMs. Sparse MoE allows models to have many more total parameters (and thus more capacity) without proportionally increasing inference cost — for example, Mixtral 8x7B has ∼47B total parameters but activates only ∼13B per token by routing to 2 of 8 experts. Unlike random forest trees, MoE experts *specialize*: the gating network learns to send different types of inputs to different experts.

A key challenge in MoE training is **expert collapse**, where the gating network funnels most inputs to one or two experts while the rest go unused, wasting capacity. This is typically addressed with an auxiliary *load-balancing loss* that penalizes uneven routing.

[4 points] In 3–5 sentences, compare how random forests and MoE each handle the tradeoff between model capacity and computational cost. Why can a random forest not "scale up" in the way that MoE can?

Solution:

1(a). Let $Z_i \in \{0,1\}$ indicate whether tree i is correct, so $\bar{Z} = X/B$ and $\mathbb{E}[\bar{Z}] = p$. The ensemble errs when at most half the trees are correct, i.e. $X \le B/2$, equivalently $\bar{Z} \le 1/2$. Take $t = p - \frac{1}{2} > 0$. Then $\bar{Z} - p \le -(p - \frac{1}{2})$ implies $\bar{Z} \le 1/2$. Hoeffding with $a_i = 0$, $b_i = 1$ gives $\sum_i (b_i - a_i)^2 = B$, hence

$$P\Big(\bar{Z} - p \le -\Big(p - \tfrac{1}{2}\Big)\Big) \le \exp\left(\frac{-2B^2(p - \frac{1}{2})^2}{B}\right) = \exp\Big(-2B\big(p - \tfrac{1}{2}\big)^2\Big)$$

1(b). For $p = 0.6$, the exponent is $-2B(0.1)^2 = -0.02B$:

$$B = 10: \ e^{-0.2} \approx 0.82, \quad B = 100: \ e^{-2} \approx 0.14, \quad B = 1000: \ e^{-20} \approx 2.1 \times 10^{-9}$$

We need $\exp(-0.02B) < 10^{-6}$, i.e. $B > \frac{\ln 10^6}{0.02} \approx 690.8$, so the first integer is $B = 691$. The bound is usually loose (conservative) because it only uses boundedness, not the binomial structure, but it still shows error probability drops exponentially in B under independence.

1(c)(i). Correlated trees reuse the same mistakes: their errors cooccur, so the average does not concentrate as if we had B independent bits of information.

1(c)(ii). Random subsets of features make splits in different trees depend on different dimensions, reducing pairwise correlation between trees so the ensemble mean behaves more like an average of weakly dependent (closer to independent) classifiers.

2(a). Let $S = \sum_b T_b$. Using $\mathrm{Cov}(T_i, T_j) = \rho\sigma^2$ for $i \ne j$ and $\mathrm{Var}(T_i) = \sigma^2$,

$$\mathrm{Var}(S) = \sum_i \mathrm{Var}(T_i) + \sum_{i \ne j} \mathrm{Cov}(T_i, T_j) = B\sigma^2 + B(B-1)\rho\sigma^2$$

Thus

$$\mathrm{Var}\left(\frac{S}{B}\right) = \frac{\sigma^2}{B} + \frac{(B-1)\rho\sigma^2}{B} = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B}$$

2(b)(i). As $B \to \infty$, the variance tends to $\rho\sigma^2$. If $\rho > 0$, we cannot drive variance to zero with more trees alone, only reducing correlation removes the floor.

2(b)(ii). If $\rho = 0$, trees are uncorrelated and we recover the usual $O(1/B)$ averaging benefit (idealized bagging with independent bootstraps). If $\rho = 1$, all trees are identical and averaging does nothing. Variance stays σ^2, as with a single tree copied B times.

2(c). $m = 1$ minimizes correlation but also starves each split of useful features, hurting bias and tree quality. The hyperparameter m trades off correlation (diversity) against the chance that each split picks informative coordinates.

3. A random forest is dense, every tree evaluates every input, so doubling trees roughly doubles inference cost, capacity scales together with compute. A sparse MoE activates only a few experts per token, so total parameters (capacity) can grow large while per token FLOPs stay controlled. Random forests cannot hide most of their capacity from each prediction the way routed experts can, so they do not scale capacity and compute independently in the same sense as MoE.

Name: Mert Iravul
Collaborators and Resources: